

A COMPLETE PROJECT MANUAL

VisionCart

A prescription eyewear shop with virtual try-on

What the project is, how much of it is finished, what is still missing, which technologies it uses, and how it is put together.

Written for a reader with no programming background,
with full technical detail in the later parts.

Version 0.1.0 · Assessed 24 August 2026

Based on inspection of the running application and all 77 source files

Contents

I	The project in plain language	4
1	What this project is	5
1.1	The one-sentence version	5
1.2	The problem it solves	5
1.3	The two halves of the product	5
1.4	Who it is for	5
2	How it works, step by step	6
2.1	A customer's journey	6
2.2	A day in the back office	6
2.3	Why the prescription rules are so strict	7
2.4	What happens to the customer's photograph	7
II	How much is finished, and what is not	8
3	How this assessment was made	9
4	What is finished and verified working	10
4.1	Verified in a live browser session	10
4.2	Depth of the underlying implementation	11
5	What is not built	12
5.1	Gaps the project itself declares	12
5.2	Further gaps found during this inspection	12
6	Completion assessment	14
6.1	Module by module	14
6.2	Overall	15
6.3	What to do next, in order	15
III	The technology stack	16
7	What the project is built from	17
7.1	In plain language first	17
7.2	Notable choices and why they were made	18
7.2.1	Next.js App Router with server components	18
7.2.2	Server actions instead of a REST API	18
7.2.3	A deliberately portable database schema	18
7.2.4	The face model is served by the project, not by Google	18
IV	Architecture	19
8	The shape of the system	20
8.1	The layers	20

8.2	Directory map	20
8.3	Size of the codebase	21
9	The data model	22
9.1	The 27 tables, grouped	22
9.2	The relationships that carry the design	23
9.3	Three invariants the schema enforces	23
10	How a request travels through the system	24
10.1	Reading a page	24
10.2	Changing something	24
10.3	Placing an order, in detail	24
11	The virtual try-on engine	26
11.1	The idea in one paragraph	26
11.2	The mathematics	26
11.3	Measuring pupil distance without a ruler	26
11.3.1	Knowing when not to trust the number	26
11.4	Graceful degradation	27
11.5	The calibration workflow	27
12	Money, pricing and discounts	28
12.1	The integer rule	28
12.2	One place decides the price	28
12.3	The discount engine	28
13	Provider adapters	29
14	Security, privacy and the audit trail	30
14.1	Sessions and passwords	30
14.2	Four roles	30
14.3	The audit trail	30
14.4	Patient data provisions already in place	30
14.5	What is still required before going live	30
V	Running and operating the system	32
15	Installing and starting it	33
15.1	First run	33
15.2	The demonstration accounts	33
15.3	Commands	34
15.4	Configuration	34
16	Moving to production	35
A	Route reference	36
A.1	Storefront	36
A.2	Back office	36
A.3	Web API endpoints	36
A.4	Known broken links	37
B	The domain library	38

C	Glossary for the non-technical reader	39
D	Summary card	40

Part I

The project in plain language

What this project is

1.1 The one-sentence version

In one sentence

VisionCart is a complete website for an opticians' shop: customers can see how a pair of glasses looks on their own face before buying, enter their eye prescription, and order lenses made to it — while the shop's staff run the entire business, from stock to patient records to the lens lab, from a private back-office area of the same site.

1.2 The problem it solves

Buying spectacles online has always been awkward for two reasons.

1. **You cannot see how they look on you.** A photograph of a frame on a white background tells you almost nothing about whether it suits your face or fits your head.
2. **Spectacles are a medical product.** They are made to a prescription written by an eye specialist. Get one number wrong and the customer gets a headache, and the shop pays to have the lenses cut again. A normal online shop — the kind that sells shoes — has no idea what any of that means.

VisionCart answers both. It puts a **virtual mirror** on the website that places the frame on the customer's own photograph at the correct size, and it treats every customer as a **patient with a file**, the way a real opticians' practice does.

1.3 The two halves of the product

The software is really two applications sharing one database.

The shop (public)	The back office (staff only)
What a customer sees. Browse frames, try them on, choose lenses, enter a prescription, pay, and track the order.	What the shop's own people see. Stock, photographs, patient files, prescriptions, orders, the lens lab queue, discounts, and settings.
13 pages	16 pages
Anyone can visit	Requires a staff login

1.4 Who it is for

- **The customer** — anyone who needs glasses and would rather not spend an afternoon in a shop.
- **Shop staff** — take orders, manage stock, upload product photographs, chase the lab.
- **The optician** — the qualified person who checks that every prescription a customer typed in is sensible before lenses are cut.
- **The owner or administrator** — prices, discounts, staff accounts, and the shop's rules.

How it works, step by step

2.1 A customer's journey

1. **Browsing.** The customer opens the shop and sees the frames. They can narrow the list by who it is for, the shape, the material, the brand, the size, or the price.
2. **The virtual mirror.** They upload a photograph of their face, or switch on their camera. The website finds their eyes and places the chosen frame over the photograph at the right size and angle. They can flick through every frame in the shop and watch each one land on their face.
3. **The measurement.** While it is doing that, the website measures the distance between the customer's pupils — an essential number for making lenses, which most people do not know. It is honest about the accuracy and always says an optician will confirm it.
4. **Choosing lenses.** The customer walks through six questions: what the glasses are for, the lens type, the thickness, the coatings, the tint, and any extras. Each answer shows what it adds to the price. The shop currently offers 24 different lens choices.
5. **The prescription.** They either type it in — using drop-down menus only, never a free-text box — or upload a photograph of the paper prescription for the optician to read.
6. **Paying.** They fill in a delivery address, pick a delivery speed and a payment method, and place the order. They do not need an account.
7. **Afterwards.** They get an order page they can return to. If they made an account, their prescription is stored and can be reused next time.

2.2 A day in the back office

1. The **dashboard** opens on the numbers that matter today: orders taken, money received, prescriptions waiting to be checked, and which frames are running low.
2. The **optician** works through the prescription queue. Each one is either verified or rejected. Nothing goes to the lens lab until a qualified person has approved it.
3. Staff open each **order**. Every order line carries its own *lab ticket*: the exact prescription, the lens choices, and a stage marker that moves from *pending* through *surfacing*, *coating* and *glazing* to *ready*.
4. Payment is confirmed, a courier and tracking number recorded, and the order is marked shipped.
5. In quieter moments: new frames are added, a photography shoot is dragged into the media library, prices are adjusted, and discounts are created — all of which take effect on the shop immediately.

2.3 Why the prescription rules are so strict

The most important design decision in the whole project

Every prescription number is chosen from a drop-down list, never typed. Eye prescriptions move in steps of 0.25 — −2.00, −2.25, −2.50 and so on. If a customer could type freely, someone would eventually enter −2.13, which no lens laboratory can make. The order would be accepted, the lenses cut wrong, and the shop would pay to remake them. Removing the text box removes the whole class of error.

Three further rules follow from the same thinking:

- A prescription is **never edited once it has been used**. A new version is created instead, so an invoice from three years ago still reads correctly.
- Each order line keeps its **own frozen copy** of the prescription, independent of the patient's file.
- A “cylinder” value without an “axis” value, or the reverse, is **rejected**, because one is meaningless without the other.

2.4 What happens to the customer's photograph

This is worth stating plainly because it is unusual.

Photographs stay on the customer's own computer

The face-detection software runs *inside the customer's web browser*. The photograph and the camera feed never travel to the shop's server. The only image that can ever reach the server is the finished picture the customer explicitly presses “save” on — and even that is switched **off** by default. With it off, the server refuses such uploads outright rather than merely hiding the button. Customers can still download their own picture either way.

Part II

How much is finished, and what is
not

How this assessment was made

The figures in this part are not taken from the project's own documentation. They come from three checks carried out directly on this copy of the project:

1. **The application was started and used.** The development server was launched and every public page and every back-office page was opened in a real browser and exercised.
2. **Every source file was read.** All 77 TypeScript files, the database schema, the seed data and the two asset scripts.
3. **The automated checks were run.** The TypeScript compiler and the code linter were both run over the whole project.

Headline result

The project builds cleanly, runs, and does what it claims to do. The type checker reports **zero errors** and the linter reports **zero warnings**. Everything the shop needs in order to take an order and fulfil it is present and working. What is missing is the ring of supporting machinery a live business needs around that core — chiefly email, password recovery, abuse protection, and automated tests.

What is finished and verified working

4.1 Verified in a live browser session

Each item below was opened and used in the running application, not merely read in the source code.

Area	What was confirmed
Home page	Renders with a live promotional banner, a three-step explainer, and a “popular this season” grid drawn from the database.
Frame catalogue	10 frames in 30 colour variants, all with generated artwork. Filters for search, wearer, shape, rim, brand, collection, size and sort all render. Sale badges and colour swatches correct.
Product page	Full specification table, four colour thumbnails, and the complete lens builder: 24 options across 6 groups, each showing its price supplement. Prescription drop-downs present with the full 0.25 range.
Virtual try-on	The studio loads and offers all 30 frames. The face-detection model initialised successfully in the browser, proving the 3.6 MB model and the 34 MB WebAssembly runtime are served correctly from the project itself rather than from an outside network.
Shopping bag	A frame was added to the bag successfully. Quantity controls, removal and the bag summary all work.
Discounts	The code WELCOME15 was applied and correctly reduced a Rs. 6,500 frame by Rs. 975, exactly 15%, producing a Rs. 5,825 total after Rs. 300 delivery. The arithmetic is exact, with no rounding drift.
Checkout	Guest checkout renders with contact fields, address, two delivery speeds priced from the rate table, and two payment methods. The card option was correctly hidden because no card-processor key is configured.
Access control	Visiting the back office, the customer account area or the checkout without a session correctly redirects to the sign-in page in every case tested.
Back office, dashboard	Eight live figures, a prescription queue, recent orders and a low-stock table, all reading real data.
Back office, orders	Order list with status and payment filters. The order detail page shows the lab ticket with the frozen prescription, a lab-stage selector, a full price breakdown, payment controls and courier controls.

Area	What was confirmed
Back office, patients	Patient file with contact details, clinical notes, consent flag, pupil measurements, the full prescription history, and verify/reject controls. A new prescription can be added from the same screen.
Back office, frames	Complete product editor: identity, style, six measurements, pricing, selling rules, search listing, plus colourway management showing stock and try-on calibration status per colour.
Back office, lenses	Every lens option listed by group with its price, prescription limits, default flag and live flag, each editable.
Back office, promotions	Four seeded deals listed with code, type, value, date window and usage, each editable.
Back office, media	Drag-and-drop library with batch tagging and search.
Back office, import	Four spreadsheet exports and two imports, with a documented column reference, a compulsory dry-run check, and a history of past import jobs including a deliberately broken file that was correctly reported as “1 ok, 2 failed”.
Back office, settings	Store details, selling rules and three try-on switches, plus a read-only panel confirming which integrations are live.

4.2 Depth of the underlying implementation

Several parts go considerably further than a demonstration would require.

- **Payments** are genuinely implemented for all three methods, not stubbed. Cash on delivery and bank transfer are complete; the card route creates a real hosted checkout session, confirms the order from the provider’s webhook rather than from the customer’s browser returning, and can issue real refunds.
- **Shipping** has three implementations. The default reads a rate table. The two live-carrier options make real calls to their providers and can buy a real printable label — and both fall back to the rate table if the key is absent or the call fails.
- **Image storage** works locally with no setup, and has a complete cloud implementation that loads its cloud library only when actually used. Every upload is rotated from its embedded orientation, capped at 2000 pixels, converted to a modern format and thumbnailed.
- **Spreadsheet handling** is written from scratch and correctly handles quoted fields, embedded commas and line breaks, doubled quotes, Windows line endings, and the invisible marker Excel puts at the start of a file.
- **Auditing** is wired into 26 places across the codebase. Every change to a patient record, a price or an order is recorded with who did it and from where.

What is not built

5.1 Gaps the project itself declares

The project’s own documentation is honest about four omissions. All four were confirmed by searching the source code.

Gap	Status	What it means in practice
Order emails	Missing	The configuration file has a mail setting, but there is not one line of code anywhere in the project that sends an email. A customer who orders receives no confirmation. This is the single most important gap.
Password recovery	Missing	A customer or member of staff who forgets their password cannot recover it. There is no “forgotten password” link and no reset mechanism.
Abuse protection	Missing	Nothing limits how many times the sign-in form or the file-upload endpoints can be hit. Passwords could be guessed by brute force, and the upload routes could be flooded.
Automated tests	Missing	There are no tests of any kind and no test framework installed. The two most obviously testable parts, the try-on geometry and the discount engine, are deliberately written as self-contained mathematics precisely so they can be tested, but nobody has written the tests.

5.2 Further gaps found during this inspection

These are not mentioned in the project’s own documentation.

Gap	Severity	Detail
Four broken links	Customer-visible	The footer of every public page links to four guide pages: how to read a prescription, measuring your pupil distance, returns, and privacy. None of the four exists; all return “page not found”.
No error pages	Cosmetic	The project defines no custom “not found” or “something went wrong” screens, so those four broken links land on the framework’s bare default page rather than on the shop’s own design.
Appointments	Unused	The database has a full appointments table — eye tests, fittings, collections, adjustments, follow-ups — and not a single line of code anywhere reads or writes it. It is a designed-but-unbuilt feature.

Gap		Severity	Detail
No audit viewer		Operational	The audit trail is written faithfully in 26 places but there is no screen anywhere that reads it back. To answer “who changed this price?” somebody must query the database directly.
No delivery-rate editor		Operational	Delivery prices live in a database table that only the code reads. The documentation states these are editable in the back office, but no such screen exists; the settings page only displays which provider is active. To change a delivery price today you must edit the database.
No address book		Minor	Addresses are saved at checkout but there is no screen where a customer can review, edit or delete them afterwards.
Cloud file deletion		Minor	When a picture is deleted and the shop is using cloud storage, the database record goes but the file itself is deliberately left in place. This is intentional and marked as such, but it means cloud storage grows forever without a manual clean-up.
Search sensitivity	case-	Minor	Catalogue search ignores capital letters on the development database but will respect them if the shop moves to PostgreSQL. Noted in the code; searching for “ravi” would then not find “Ravi”.

Completion assessment

6.1 Module by module

The percentages below are a judgement, not a measurement. They express how much of what that area would need *for a real shop to open* is present.

Area	Assessed	Basis
Database design	100%	27 tables, fully related, deliberately portable across three database engines. Consent and data-retention fields already present.
Virtual try-on	95%	Complete and working, including the manual fallback when detection fails. Only real photography of real frames is missing.
Product catalogue	95%	Browsing, filtering, sorting, colourways, stock, images and calibration are all done.
Lens builder	95%	Six-step wizard, 24 options, cross-option rules and prescription-based limits all enforced.
Prescriptions	95%	Versioning, verification queue, per-order snapshots, and strict validation. Reading an uploaded paper prescription is still a human job, as intended.
Shopping bag and checkout	90%	Complete and re-priced from the database on every read. Missing only the confirmation email.
Discount engine	95%	Six kinds of deal with nine conditions and controlled stacking. Verified working live.
Payments	90%	Three methods fully implemented. Untested against live provider keys in this environment.
Shipping	85%	Three providers implemented with safe fallback, but the rate table has no editing screen.
Back office	90%	Nine screens, all functional. Missing an audit viewer and a rate editor.
Images and media	95%	Bulk upload, processing, thumbnails, two storage back-ends.
Import and export	90%	Four exports, two imports, dry-run and job history.
Accounts and access	70%	Sign-in, registration, four roles and guards on every route are done. No password recovery and no abuse protection.
Audit and privacy	75%	Recording is thorough and thoughtfully designed. It cannot be read back, and encryption at rest is not configured.
Notifications	5%	A configuration setting exists. Nothing else.

Area	Assessed	Basis
Public content pages	0%	Four pages are linked from every page and none of them exists.
Automated testing	0%	None.

6.2 Overall

Roughly 85% complete as a product; approximately 70% ready to open to the public

The distinction matters. As a *piece of software* VisionCart is close to finished: the difficult, distinctive work — the try-on mathematics, the clinical data rules, the pricing engine, the provider adapters — is all done and demonstrably works. As a *live business* it is not yet ready, because a shop that cannot email a customer, cannot reset a forgotten password, cannot resist a password-guessing attack, and shows four broken links on every page is not one you would open to the public. Those four items are small pieces of work compared with what has already been built — realistically a week or two — but every one of them is compulsory.

6.3 What to do next, in order

1. **Send emails.** Order confirmation, dispatch notice, and prescription-verified notice. Attach to the order-placing code and the payment webhook.
2. **Write the four missing guide pages**, or remove the links. This is an afternoon's work and it is currently visible to every visitor.
3. **Add password recovery.**
4. **Add rate limiting** to sign-in and to the upload routes.
5. **Write tests** for the try-on geometry and the discount engine first; both are pure mathematics and easy to test.
6. **Add an audit-log viewer** and a delivery-rate editor to the back office.
7. **Add custom error pages.**
8. **Then** decide whether appointments are wanted; the database is already waiting for them.

Part III

The technology stack

What the project is built from

7.1 In plain language first

A modern website of this kind is assembled from a handful of well-known, freely available building blocks. Nobody writes an online shop from nothing. What follows is the list of blocks this project uses, why each one is there, and — for the non-technical reader — what job it actually does.

Component	Version	What it does, in plain terms
Next.js	16.3.1	The framework that holds everything together. It decides which page to show for which web address, runs code on the server, and sends finished pages to the browser.
React	19.2.8	The library that draws the interface and keeps it in step with the data. Everything the user sees is a React component.
TypeScript	5.x	JavaScript with type checking added. It catches whole classes of mistakes before the code ever runs, which is why the project can report “zero errors”.
Tailwind CSS	4.x	The styling system. Appearance is written as short utility names directly alongside the markup rather than in separate stylesheets.
Prisma	6.19.3	The translator between the program and the database. The developer describes the tables once; Prisma generates the code to read and write them safely.
SQLite	bundled	The database itself, in development. It is a single file on disk requiring no installation, which is why the project runs immediately after download.
MediaPipe Tasks Vision	1.0.1	Google’s face-analysis library. It finds 478 points on a face, including the irises. Critically, it runs <i>inside the browser</i> , which is what keeps customer photographs private.
sharp	0.35.3	Image processing on the server: rotating photographs the right way up, shrinking them, converting them and making thumbnails.
jose	6.2.9	Creates and verifies the signed token that proves a visitor is signed in.
bcryptjs	3.0.3	Scrambles passwords so that even someone who steals the database cannot read them.
Zod	4.4.3	Checks that data arriving from a form or a spreadsheet is the right shape before anything is done with it.

Component	Version	What it does, in plain terms
Stripe	22.5.0	The card-payment provider's toolkit. Loaded only if card payments are switched on.
ESLint	9.x	Automated style and correctness review of the code.
tsx	4.23.12	A small helper used to run the database seeding script.

7.2 Notable choices and why they were made

7.2.1 Next.js App Router with server components

Most of this application's pages are rendered on the server, which means the browser receives finished HTML rather than a program that then has to fetch data and draw itself. Only the genuinely interactive pieces — the try-on canvas, the lens builder, the file uploader, the forms — are shipped to the browser as code. The practical result is that catalogue pages appear quickly even on a slow connection.

7.2.2 Server actions instead of a REST API

Rather than building a separate web API and then calling it from the browser, the project uses Next.js *server actions*: functions marked as server-side that a form can call directly. There are 32 of them. Web API routes exist only where server actions genuinely cannot be used — file uploads, spreadsheet download and upload, and the payment provider's webhook.

7.2.3 A deliberately portable database schema

The database description avoids three features that most projects use freely: enumerated types, array columns and native JSON columns. This is not an oversight. Those three features are the usual reason a project written against one database cannot be moved to another. By avoiding them, the same schema runs unchanged on SQLite, PostgreSQL and MySQL, and moving to a production database is a one-line change.

The cost is that fields which *would* be enumerations — order status, lens group, prescription status — are plain text, validated in code instead. All the permitted values live in one file, `src/lib/constants.ts`, which is the single source of truth for validation, drop-down menus and labels alike.

7.2.4 The face model is served by the project, not by Google

The face-detection model and its runtime are 37 MB of files copied into the project's own public folder. They could have been loaded from Google's content network with one line of configuration. They are not, for two reasons: no third party ever learns that a particular visitor is using the try-on, and the feature keeps working in networks where Google's content network is unreachable.

Part IV

Architecture

The shape of the system

8.1 The layers

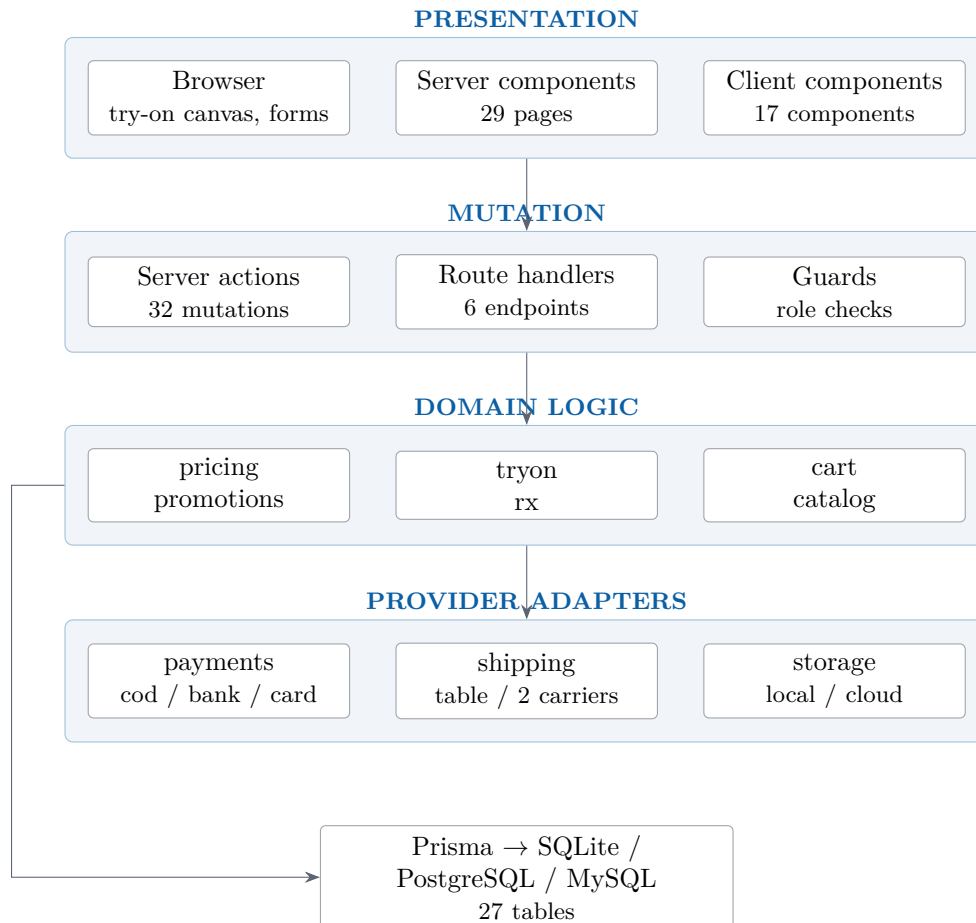


Figure 8.1: The five layers. Each one may call downwards, never upwards.

8.2 Directory map

```

prisma/
  schema.prisma 27 models, portable across three databases
  seed.ts staff, catalogue, lenses, rates, promotions, demo data
  migrations/ one migration, 19.8 KB of SQL

scripts/
  fetch-tryon-assets.mjs copies the WebAssembly runtime, downloads the model
  generate-frame-assets.mjs draws placeholder frame artwork

public/
  models/ face_landmarker.task 3.6 MB
  wasm/ MediaPipe runtime 34 MB
  frames/ 31 generated overlays 1.1 MB
  uploads/ locally stored images
  
```

```
src/  
  app/  
    (shop)/ 13 storefront pages  
    admin/ 16 back-office pages  
    actions/ 32 server actions in 4 files  
    api/ 6 route handlers  
  components/  
    tryon/ the virtual mirror (3 files, 854 lines)  
    shop/ 8 storefront components  
    admin/ 7 back-office components  
  lib/ 17 modules -- the domain layer
```

8.3 Size of the codebase

Measure	Count
TypeScript and TSX source lines	13,676
Schema, seed data and asset scripts	1,618
Source files	77
Database tables	27
Storefront pages	13
Back-office pages	16
Web API endpoints	6
Server actions	32
React components	17
Domain library modules	17
Audit call sites	26
Type errors	0
Lint warnings	0

The data model

9.1 The 27 tables, grouped

Group	Tables	Purpose
People and access	User, Patient, Prescription, PatientDocument, Appointment, Address	The clinical record. Every customer, whether registered or a guest, gets a patient file with a sequential number such as P-000042.
Catalogue	Brand, Category, FrameCategory, Frame, FrameVariant, ProductImage	A frame is a model; a variant is one colour-way of it, carrying its own stock, barcode, price override and try-on calibration.
Lenses	LensOption	One table drives the entire six-step lens wizard, its prices, its prescription limits and its compatibility rules.
Commerce	Cart, CartItem, Order, OrderItem, Payment, Shipment, ShippingRate	Baskets, orders, money and parcels. Order lines carry frozen snapshots of everything that mattered at the moment of sale.
Marketing	Promotion	Six kinds of deal, each with nine possible conditions.
Virtual try-on	TryOnSession, TryOnSnapshot	Only ever written when a signed-in customer explicitly saves a picture and the shop has that feature enabled.
Operations	Setting, MediaAsset, ImportJob, AuditLog	Shop configuration a manager can change without a developer; the image library; spreadsheet job history; and the trail of who changed what.

9.2 The relationships that carry the design

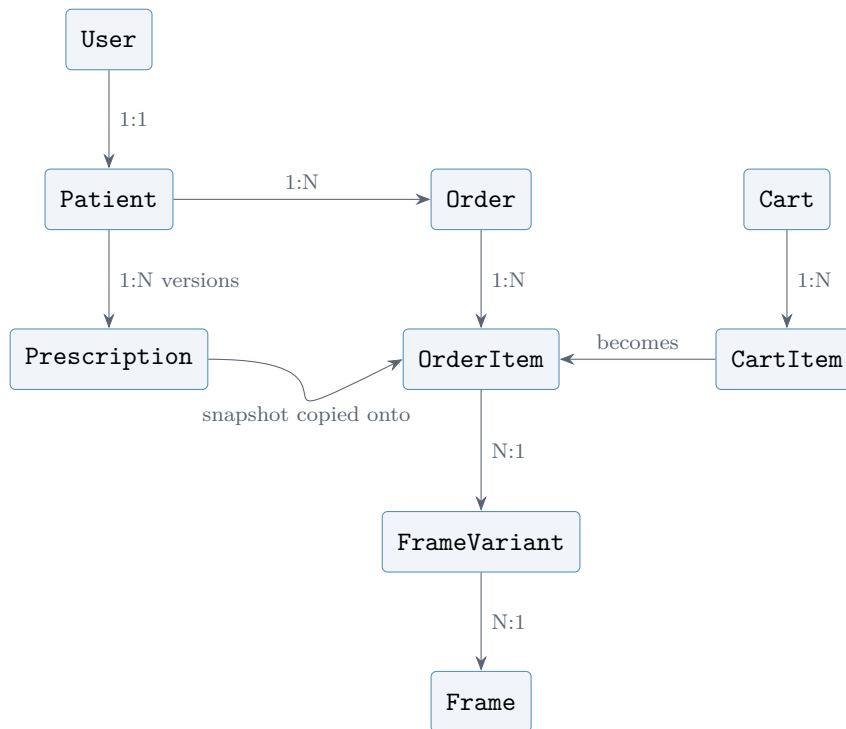


Figure 9.1: The spine of the model. Note the curved relationship: an order line does not merely *point at* a prescription, it also stores its own frozen copy of it.

9.3 Three invariants the schema enforces

1. **Everyone has a patient file.** Registration creates one automatically. So does a guest checkout — matched on email address so a returning guest keeps the same file rather than accumulating new ones. An optical order that cannot be traced to a patient cannot be remade or followed up.
2. **Prescriptions are versioned, never overwritten.** Statuses run *draft*, *pending verification*, *verified*, *rejected*, *expired*. A customer-entered prescription always begins as *pending verification* and appears in the optician’s queue. The lab ticket refuses to look ready until a qualified person has verified it.
3. **Order lines are self-contained history.** Each line stores its own title, SKU, image, unit price, lens price, lens option codes, a lens summary and a complete frozen prescription including the pupil distance. An invoice from three years ago still reads correctly even if the product has since been renamed, re-priced or deleted.

How a request travels through the system

10.1 Reading a page

1. The browser asks for a web address, for example `/frames/ravi`.
2. Next.js matches it to a server component, which runs on the server.
3. That component calls a library module in `src/lib` — `catalog.ts` for the product, `pricing.ts` for the price.
4. The library module queries the database through Prisma.
5. Finished HTML is streamed to the browser. Only the interactive parts — the lens builder and the try-on canvas — also send code.

10.2 Changing something

1. The user submits a form.
2. The form calls a *server action* directly. No hand-written web API sits in between.
3. The action first runs its guard. Staff actions call `staff()`; pages call `requireStaff()` or `requireAdmin()`. All 20 back-office actions were verified to carry a guard.
4. The action validates its input with Zod.
5. It performs the change, wrapping multi-table work in a database transaction.
6. It writes an audit entry.
7. It tells Next.js which cached pages are now stale so the screen refreshes.

Why route handlers still exist

Server actions cannot stream a large file upload, cannot return a downloadable spreadsheet, and cannot receive a signed webhook from a payment provider. Those three cases — and only those three — use the six conventional web API endpoints under `src/app/api`.

10.3 Placing an order, in detail

This is the most complex path in the system. It is worth following because almost every design decision in the project shows up in it.

1. The submitted form is validated against a schema.
2. The chosen payment method is checked against the methods the shop has actually enabled — a tampered form cannot select a disabled one.
3. **The entire basket is re-priced from the database.** Nothing the browser sent about price is trusted.
4. Out-of-stock lines block the order.

5. If the shop requires a prescription on every pair, every line is checked for one.
6. Discounts are re-evaluated from scratch.
7. Delivery is re-quoted for the address actually entered.
8. Totals are computed: goods, minus discount, plus delivery, plus tax.
9. A patient file is found or created.
10. A sequential order number is allocated, in the form VC-2026-000001.
11. **Inside one database transaction:** the address is stored, the order is created, and for each line — any prescription typed at checkout is promoted into a real versioned record marked *pending verification*, the order line is written with its frozen snapshot, and stock is decremented.
12. The basket is emptied but the row is kept for analytics.
13. An audit entry is written.
14. Payment is started. If the payment provider fails, the order is *not* discarded — it is marked payment-failed with a note, so staff can take payment by hand rather than lose the sale.
15. A shipment record is created and the customer is sent either to the provider's payment page or to their order page.

The virtual try-on engine

11.1 The idea in one paragraph

A frame overlay is a transparent picture plus two *anchors*: the points inside that picture where the wearer’s pupils ought to sit. Given the customer’s two pupils in their photograph, there is **exactly one** similarity transform — a uniform scale, a rotation and a shift — that puts the anchors on the pupils. That single fact is the entire placement algorithm. There is no per-face tuning, no machine learning at drawing time, and nothing to go subtly wrong.

11.2 The mathematics

Let a_L, a_R be the anchor points expressed in the overlay’s own pixels, and p_L, p_R the customer’s pupils in canvas pixels. Write k for a per-artwork correction factor and s_u, θ_u, Δ_u for the customer’s own manual nudges.

$$\text{scale} \quad s = \frac{\|p_R - p_L\|}{\|a_R - a_L\|} \cdot k \cdot s_u \quad (11.1)$$

$$\text{rotation} \quad \theta = \text{atan2}(p_{R_y} - p_{L_y}, p_{R_x} - p_{L_x}) - \text{atan2}(a_{R_y} - a_{L_y}, a_{R_x} - a_{L_x}) + \theta_u \quad (11.2)$$

$$\text{translation} \quad t = p_L + \Delta_u \quad (11.3)$$

The overlay is then drawn by translating to t , rotating by θ , scaling by s , and translating back by $-a_L$.

11.3 Measuring pupil distance without a ruler

Pupil distance — the separation between the centres of the two pupils — is essential for making lenses and almost nobody knows their own. The trick used here is that the visible iris of an adult eye is 11.7 mm across and varies remarkably little between people. It therefore works as a ruler that is already in the photograph.

$$\text{PD}_{\text{mm}} = \|p_R - p_L\|_{\text{px}} \times \frac{11.7}{\bar{d}_{\text{iris,px}}} \quad (11.4)$$

where $\bar{d}_{\text{iris,px}}$ is the mean of the two measured iris widths. Expect roughly ± 2 mm.

11.3.1 Knowing when not to trust the number

The system reports a confidence score rather than a bare figure:

$$c = \text{clamp}_{[0,1]} \left(1 - 2.5\alpha - 0.5 \min \left(1, \frac{|\rho|}{25} \right) \right), \quad \alpha = \frac{|d_L - d_R|}{\max(d_L, d_R)} \quad (11.5)$$

Here α is the asymmetry between the two measured irises and ρ is the head roll in degrees. Irises of visibly different width mean the head is turned away from the camera, which breaks the assumption that both pupils lie in one plane. A prescription of implausible width — below 48 mm or above 80 mm — has its confidence capped at 0.2 rather than being reported as fact.

An honest measurement

The number is only written to the patient's file when confidence reaches 0.5 *and* no optician has already recorded a pupil distance by hand. A machine estimate never overwrites a human measurement. The customer is always told an optician will confirm it.

11.4 Graceful degradation

If the browser is too old, the model file is missing, or the photograph simply defeats detection, the studio does not fail. It reports the detector as unavailable and lets the customer drag two markers onto their own eyes. Everything downstream — placement, measurement, saving — is then identical. The feature degrades; it never breaks.

11.5 The calibration workflow

Adding artwork for a real frame is a back-office task:

1. Open the frame, then a colourway, then the virtual-try-on tab.
2. Upload a front-on transparent picture with the temples pointing out.
3. Drag the **L** and **R** markers onto the centre of each lens.
4. Check the live preview against a reference face and save.

Colourways without artwork simply do not appear in the mirror, and the frame page warns staff which ones are missing. The placeholder artwork shipped with the project is drawn by a script so that a fresh installation has a working shop before any photography has been done.

A coupling worth knowing about

The default anchor positions live in `src/lib/tryon.ts` and the matching geometry constants live in `scripts/generate-frame-assets.mjs`. They are two halves of one contract. Changing one without the other will misplace every generated frame.

Money, pricing and discounts

12.1 The integer rule

Money is never a decimal number in this system

Every monetary value is stored and calculated as a whole number of the smallest unit — paisa, or cents. Rs. 6,500.00 is stored as the integer 650000. Staff type ordinary amounts into forms and the conversion happens exactly once, at the edge. Decimal arithmetic on money is how a shop ends up charging Rs. 0.01 less than it should on thousands of orders.

There is one further subtlety: the display formatter deliberately avoids the browser’s built-in currency formatting, because that inserts a space whose width differs between server and browser and causes the page to flicker on load.

12.2 One place decides the price

Every price in the application is computed by `src/lib/pricing.ts`. No component computes a total. The basket is re-priced from the database on *every single read*, which produces two useful properties:

- A tampered payload from the browser cannot change what is charged.
- A price change or a stock change is reflected the instant the customer looks at their bag, with no cache to invalidate.

The same module also rejects lens combinations the laboratory cannot make — options that require each other, options that exclude each other, and options whose prescription limits the customer’s own prescription exceeds. Problems are returned as readable sentences rather than thrown as errors, so the interface can show them beside the choice that caused them.

12.3 The discount engine

Six kinds of deal are supported: percentage off, fixed amount off, free delivery, buy-one-get-one, free lens upgrade, and bundle price. Each can be restricted by nine conditions: minimum spend, minimum quantity, brand, collection, specific frames, a date window, a total usage cap, a per-customer cap, and first-orders-only.

The stacking rule

The highest-priority offer always applies. Any other offer joins it only if *every* promotion currently in play is marked stackable. The effect is that a headline deal never silently doubles up, but a delivery perk can ride alongside a discount code.

Two details are worth noting. The buy-one-get-one calculation expands the basket into individual units, sorts them by price and makes every second one free from the cheapest end — so the customer always keeps the more expensive item. And when a customer types a code that does not apply, the engine explains *why* in words they can act on: expired, does not apply to anything in the bag, needs more items, first orders only, or already used.

Provider adapters

Three parts of the system talk to the outside world. All three are built the same way: one interface, several implementations, chosen by an environment setting, each degrading safely when its key is absent.

Concern	Options	Behaviour
Payments	cash on delivery, bank transfer, card	Cash and bank transfer need no account and work immediately. The card option uses a hosted payment page, so card details never touch this server. Crucially the order is confirmed by the provider's <i>webhook</i> , not by the customer's browser returning — so a customer who closes the tab after paying still gets their glasses. The card option hides itself entirely if no key is configured.
Shipping	rate table, two live carriers	The rate table is the default and needs no account. The two live options fetch real carrier rates and can buy a real printable label. Both fall back to the rate table if the key is missing or the call fails, so a carrier outage degrades to flat-rate delivery instead of blocking checkout.
Image storage	local disk, cloud	Local writes into the project's own public folder with zero setup. The cloud option covers the three common providers and loads its library only when actually used, so the dependency is not required unless it is wanted.

Adding a fourth payment provider means writing one adapter function and adding one entry to a list. Nothing about any provider leaks into the checkout interface.

Security, privacy and the audit trail

14.1 Sessions and passwords

Sessions are signed tokens stored in a cookie that JavaScript cannot read, set to expire after 30 days, and marked secure in production. Passwords are hashed with bcrypt at 10 rounds. The sign-in form returns the *same* message for “no such account” and “wrong password”, so it cannot be used to discover which email addresses have accounts. The application refuses to start if the signing secret is missing or shorter than 32 characters.

14.2 Four roles

Role	Access
customer	The shop and their own account only.
staff	The back office.
optician	The back office, including prescription verification.
admin	Everything, including administrator-only screens.

Guards are applied per page and per action rather than through a single global filter. This was verified by request: the back office, the account area and the checkout all redirect an unauthenticated visitor to the sign-in page.

14.3 The audit trail

Every change to a patient record, a price or an order writes a row recording the actor, the action, the entity, the time and the client address. Even spreadsheet exports of patient data are audited, because exporting clinical data is itself an event worth recording.

A deliberate omission

Clinical values are kept *out* of the audit detail. The reasoning is that an audit log is read far more widely than the record it describes, so putting prescriptions in it would quietly widen access to health data. An audit failure also never takes down the operation it was recording — the write is wrapped so a logging problem cannot lose a sale.

14.4 Patient data provisions already in place

The `Patient` table already carries a marketing-consent flag, the date and version of consent given, a retention deadline and a soft-delete marker. The data model is therefore ready for a retention policy; the policy decisions themselves have not been made.

14.5 What is still required before going live

- Encryption of the database at rest.
- A documented retention period, using the fields already present.
- A customer-facing route for correction and erasure requests.

-
- Rate limiting on sign-in and uploads (see the gaps chapter).
 - Replacement of the seeded demonstration passwords.

Part V

Running and operating the system

Installing and starting it

15.1 First run

```
npm install
cp .env.example .env
```

Open `.env` and set `AUTH_SECRET` to a long random string. One can be generated with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Then build the database, fetch the face model, draw the placeholder artwork and load the demonstration data:

```
npm run setup
npm run dev
```

The shop appears at `http://localhost:3000` and the back office at `http://localhost:3000/admin`.

A note from this session

On the machine used for this assessment port 3000 was already occupied by another program, so the development server was started on an alternative port instead. This is normal and harmless. If it happens, the address the server prints on startup is the one to use.

15.2 The demonstration accounts

The seed data creates three accounts. Their passwords are published in the project's own documentation, which means they are effectively public.

Role	Email	Password
Owner	admin@visioncart.local	ChangeMe123!
Optician	optician@visioncart.local	Optician123!
Customer	demo@example.com	Demo1234!

Change these before the shop is reachable from the internet

Set the seeded administrator email and password in `.env` before seeding, or change the password from the back office afterwards. Leaving them in place on a public server would hand anybody full access to every patient record.

15.3 Commands

Command	What it does
<code>npm run dev</code>	Development server
<code>npm run build</code>	Production build
<code>npm run setup</code>	Migrate, fetch assets, draw artwork, seed
<code>npm run db:migrate</code>	Create and apply a database migration
<code>npm run db:seed</code>	Reload demonstration data (safe to repeat)
<code>npm run db:studio</code>	Browse the database in a window
<code>npm run db:reset</code>	Drop and rebuild the database
<code>npm run tryon:assets</code>	Re-fetch the face model and runtime
<code>npm run frames:generate</code>	Redraw the placeholder frame artwork
<code>npm run typecheck</code>	Check types across the whole project
<code>npm run lint</code>	Run the code linter

15.4 Configuration

Only two settings are required to start: the database location and the signing secret. Every integration degrades to a safe offline behaviour until real keys are supplied, which is why the whole shop can be demonstrated before signing up for anything.

Configuration is split deliberately. **Secrets and infrastructure** live in the environment file and require a developer. **Anything a shop manager should own** — store details, free-delivery threshold, return window, guest checkout, whether a prescription is compulsory, and the three try-on switches — lives in the database and is editable from the settings screen.

Moving to production

1. **Switch to PostgreSQL.** Change the provider in the schema, point the database address at the new server and deploy the migration. Because the schema avoids enumerations, arrays and native JSON, no other edit is needed.
2. **Replace every secret.** A fresh signing secret, new staff passwords, real payment keys.
3. **Set the public address correctly,** because the payment provider's return links are built from it.
4. **Move image storage to the cloud** unless the server has durable, backed-up disk.
5. **Point the payment webhook** at the project's webhook endpoint and set its signing secret.
6. **Set the tax rate,** expressed in basis points, and say whether displayed prices already include it.
7. **Complete the four compulsory gaps** listed in the completion chapter: email, the guide pages, password recovery and rate limiting.

Route reference

A.1 Storefront

Address	Page
/	Home
/frames	Catalogue with filters
/frames/[slug]	Product page and lens builder
/try-on	Virtual mirror
/deals	Current offers
/cart	Shopping bag
/checkout	Checkout
/order/[orderNo]	Order confirmation and tracking
/login, /register	Access
/account	Customer account
/account/orders	Order history
/account/prescriptions	Saved prescriptions

A.2 Back office

Address	Screen
/admin	Dashboard
/admin/orders, /admin/orders/[id]	Orders and lab tickets
/admin/patients, /admin/patients/[id]	Patient files
/admin/patients/new	New patient
/admin/frames, /admin/frames/[id]	Catalogue
/admin/frames/new	New frame
/admin/media	Image library
/admin/lenses	Lens options and prices
/admin/promotions, /admin/promotions/[id]	Deals
/admin/promotions/new	New deal
/admin/import	Spreadsheet import and export
/admin/settings	Store settings

A.3 Web API endpoints

Endpoint	Why it is not a server action
/api/admin/upload	Streams a multipart file upload
/api/admin/import	Streams a spreadsheet upload
/api/admin/export	Returns a downloadable file
/api/account/prescription-upload	Streams a file upload
/api/tryon/snapshot	Streams an image upload
/api/webhooks/stripe	Receives a signed external callback

A.4 Known broken links

Address linked from every page footer	Result
/guides/prescription	404 not found
/guides/pd	404 not found
/guides/returns	404 not found
/guides/privacy	404 not found

The domain library

The seventeen modules in `src/lib` are where the actual thinking lives.

Module	Lines	Responsibility
<code>tryon.ts</code>	303	Placement geometry and pupil-distance measurement. Pure functions, no browser and no server imports, so the same mathematics runs in the canvas, in a test and on the server.
<code>shipping.ts</code>	324	Three delivery providers behind one interface.
<code>payments.ts</code>	291	Three payment providers behind one interface.
<code>promotions.ts</code>	261	Deal evaluation, eligibility and stacking.
<code>pricing.ts</code>	221	The only place a price is decided.
<code>constants.ts</code>	217	Every permitted value in the system.
<code>cart.ts</code>	215	Basket identity, merging a guest bag on sign-in.
<code>catalog.ts</code>	206	Shared catalogue queries and filters.
<code>storage.ts</code>	197	Image processing and two storage back-ends.
<code>rx.ts</code>	181	Prescription validation and clinical helpers.
<code>auth.ts</code>	147	Sign-in, registration, patient files, guards.
<code>csv.ts</code>	102	Spreadsheet parsing and writing, from scratch.
<code>session.ts</code>	75	Signed session cookies.
<code>settings.ts</code>	53	Shop configuration with defaults.
<code>money.ts</code>	49	Integer money and display formatting.
<code>audit.ts</code>	39	The audit trail.
<code>db.ts</code>	14	The database connection, cached across reloads.

Glossary for the non-technical reader

Term	Meaning
Anchor	The two points marked inside a frame picture showing where the wearer's pupils should sit. Calibrating a frame means placing these two points.
Axis	The angle, from 1 to 180 degrees, at which an astigmatism correction is oriented. Meaningless without a cylinder value.
Colourway or variant	One colour of one frame model. Stock, barcode, price and try-on calibration all belong to the colourway, not the model.
Cylinder	The strength of the astigmatism correction, in the same 0.25 steps as the sphere.
Diopter	The unit of lens strength. Written with a sign and two decimals, always in steps of 0.25.
Migration	A recorded change to the shape of the database, so the same change can be applied to another copy in the same order.
Minor units	The smallest denomination of a currency: paisa, cents. All money in this system is stored as a whole number of these.
OD and OS	Latin abbreviations for the right eye and the left eye, used on every prescription in the world.
PD, pupillary distance	The distance in millimetres between the centres of the two pupils. Needed to position the optical centre of each lens.
Seeding	Loading a set of realistic starting data so a fresh installation has a working shop to look at.
Server action	A function that a form can call directly, which runs on the server. Used here instead of building a separate web API.
Sphere	The main strength of a lens. Negative for short sight, positive for long sight.
WebAssembly	A format that lets demanding code, such as face detection, run at near-native speed inside a web browser.

Summary card

VisionCart at a glance

What it is: a complete online opticians' shop with an in-browser virtual try-on and a full clinical back office.

Built with: Next.js 16, React 19, TypeScript, Tailwind 4, Prisma and MediaPipe.

Size: 13,676 lines of source across 77 files, 27 database tables, 29 pages, 32 server actions.

Health: zero type errors, zero lint warnings, runs correctly.

Completion: roughly 85% as a product, about 70% ready for the public.

Blocking work before launch: transactional email, four missing guide pages, password recovery, rate limiting.

Strongest parts: the try-on geometry, the clinical data rules, the single-source pricing engine, and the provider adapters.

Weakest parts: no tests, no notifications, no audit viewer.

This manual was produced by inspecting the running application and every source file in the project. Percentages are informed judgements; every specific claim about behaviour was verified either in the browser or in the source.